Blazor Server
For Dummies

- ✓ Simple explanations for complex concepts
- 📖 Real-world analogies you'll actually remember
- 💡 Focus on the "why" before the "how"

Don't worry if you're new to this - we all started somewhere! This guide breaks down Blazor patterns into bite-sized pieces.

What's Inside

Part I: Building Blocks

The foundations every Blazor app needs

Chapter 1: Base Classes

Don't Repeat Yourself!

> **Analogy:** Imagine you run a pizza shop. Every pizza has dough, sauce, and cheese. Would you write out "dough, sauce, cheese" on every single order? No! You'd have a "base pizza" that all pizzas start from. That's what base classes do for your code!

The Problem

Look at these three classes. Notice anything repetitive?

```
public class Board
{
    public int Id { get; set; }           // Same!
    public DateTime CreatedAt { get; set; } // Same!
    public string Name { get; set; }
}

public class Task
{
    public int Id { get; set; }           // Same!
    public DateTime CreatedAt { get; set; } // Same!
    public string Title { get; set; }
}

public class User
{
    public int Id { get; set; }           // Same!
    public DateTime CreatedAt { get; set; } // Same!
    public string Email { get; set; }
}
```

We're writing Id and CreatedAt over and over. If we have 20 classes, that's 20 copies. What if we need to add UpdatedAt later? We'd have to change ALL 20 files!

The Solution

Create a "base class" with the common stuff, then have other classes inherit from it:

```
// The "base pizza" - shared by all
public abstract class BaseEntity
{
    public int Id { get; set; }
    public DateTime CreatedAt { get; set; }
}

// Now our classes are simpler!
public class Board : BaseEntity  // Gets Id and CreatedAt for free!
{
    public string Name { get; set; }
}

public class Task : BaseEntity   // Gets Id and CreatedAt for free!
{
```

```
    public string Title { get; set; }
}
```

> **Key Point:** The word **abstract** means "you can't create a BaseEntity directly" - it only exists to be a parent for other classes. Like how "pizza" is a concept, but you always order a specific type (Margherita, Pepperoni, etc.).

Why This Rocks

* **Write once, use everywhere** - Common fields defined in one place
* **Easy changes** - Add UpdatedAt to BaseEntity, ALL classes get it
* **Consistency** - Every entity has the same basic structure

Chapter 2: Database Setup

Teaching EF Core About Your Data

> **Analogy:** Entity Framework Core (EF Core) is like a builder, and your database is the house. But the builder needs blueprints! Configuration classes are those blueprints - they tell EF Core exactly how to build your database tables.

The Three Ways to Configure

Way 1: Annotations (Quick but Limited)

```
public class User
{
    public int Id { get; set; }

    [Required]           // Can't be empty
    [MaxLength(100)]     // Max 100 characters
    public string Name { get; set; }
}
```

✓ Easy to read ✗ Limited options ✗ Clutters your class

Way 2: All in One Place (Messy)

```
// Everything crammed into one giant method
protected override void OnModelCreating(ModelBuilder m)
{
    // User config... 50 lines
    // Board config... 50 lines
    // Task config... 50 lines
    // This file becomes HUGE!
}
```

✗ One massive file ✗ Hard to find things

Way 3: Separate Config Files (The Good Way!)

```
// UserConfiguration.cs - just User stuff
public class UserConfiguration : IEntityTypeConfiguration<User>
{
    public void Configure(EntityTypeBuilder<User> builder)
    {
        builder.Property(u => u.Email)
            .IsRequired()
            .HasMaxLength(256);

        builder.HasIndex(u => u.Email)
            .IsUnique();  // No duplicate emails!
    }
}

// In your DbContext - one magic line loads ALL configs!
modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
```

✓ Organized ✓ Easy to find ✓ Clean DbContext

Key Point: Think of each configuration file as a blueprint for ONE table. UserConfiguration = User table blueprint. BoardConfiguration = Board table blueprint.

Chapter 3: ServiceResult Pattern

How to Say "It Worked!" or "Oops!"

> **Analogy:** When you order something online, you don't just get silence - you get tracking updates: "Shipped!", "Out for delivery!", or "Couldn't deliver - wrong address". ServiceResult is like tracking for your code operations!

The Problem

What if creating a user fails? How do you tell the caller what went wrong?

```
// Bad Way 1: Return null
public User? CreateUser(string email)
{
    if (EmailExists(email))
        return null;  // WHY did it fail? Who knows!
}

// Bad Way 2: Throw exceptions everywhere
public User CreateUser(string email)
{
    if (EmailExists(email))
        throw new Exception("Email exists");  // Messy!
}
```

The Solution

Return a "package" that contains both the result AND any messages:

```
public class ServiceResult<T>
{
    public bool Success { get; set; }    // Did it work?
    public T? Data { get; set; }         // The actual result
    public string? Message { get; set; } // What happened?
    public List<string> Errors { get; set; } = [];

    // Easy ways to create results:
    public static ServiceResult<T> Ok(T data) =>
        new() { Success = true, Data = data };

    public static ServiceResult<T> Fail(string error) =>
        new() { Success = false, Message = error, Errors = [error] };
}
```

Using It

```
// In your service:
public ServiceResult<User> CreateUser(string email)
{
    if (EmailExists(email))
        return ServiceResult<User>.Fail("Email already exists!");

    var user = new User { Email = email };
    // save to database...
```

```
    return ServiceResult<User>.Ok(user);
}

// In your component:
var result = await UserService.CreateUser(email);

if (result.Success)
    ShowMessage("User created!");
else
    ShowError(result.Message);  // "Email already exists!"
```

Key Point: The caller ALWAYS knows what happened. No guessing, no crashes, just clear communication!

Chapter 4: Adapter Pattern

The Universal Translator

> **Analogy:** Your US phone charger won't fit European outlets. You need an adapter! In code, when Module A speaks "User" but Module B speaks "Employee", an adapter translates between them.

The Problem

You built a Learning Management System (LMS) that works with your User class. Now another company wants to use it, but they have Employee, not User. Rewrite everything?

The Solution

Define what the LMS ACTUALLY needs (an interface), then create adapters for different sources:

```
// Step 1: What does LMS need? Just these basics:
public interface ILearner
{
    int Id { get; }
    string Name { get; }
    string Email { get; }
}

// Step 2: Adapter for YOUR system
public class UserAdapter : ILearner
{
    private readonly User _user;

    public int Id => _user.Id;
    public string Name => _user.UserName;  // Translates UserName -> Name
    public string Email => _user.Email;
}

// Step 3: Adapter for THEIR system
public class EmployeeAdapter : ILearner
{
    private readonly Employee _emp;

    public int Id => _emp.EmployeeId;      // Different field name!
    public string Name => _emp.FullName;   // Different field name!
    public string Email => _emp.WorkEmail; // Different field name!
}
```

Now the LMS only talks to ILearner. It doesn't care if it's a User, Employee, or even a Robot - as long as there's an adapter!

> **Key Point:** Adapters let you reuse code with different data sources. Build once, adapt many times!

Chapter 5: Bulk Operations

Importing Data Without Falling Asleep

> **Analogy:** Would you move 1000 boxes by carrying them one at a time to your car, driving to the new house, and coming back? No! You'd load a truck with ALL the boxes and make ONE trip. Bulk operations are the moving truck for your data!

The Slow Way

```
// One trip per item = SLOW
foreach (var item in thousandItems)
{
    context.Items.Add(item);
    await context.SaveChangesAsync();  // Trip to database #1, #2, #3... #1000!
}
// Time: ~30 seconds for 100 items
```

The Fast Way

```
// Load the truck, one trip!
// Install: EFCore.BulkExtensions NuGet package

await context.BulkInsertAsync(thousandItems, new BulkConfig
{
    SetOutputIdentity = true,      // Get the new IDs back!
    PreserveInsertOrder = true     // Keep items in order
});
// Time: ~0.5 seconds for 100 items (60x faster!)
```

The Magic Settings

Setting	What It Does
SetOutputIdentity = true	After insert, your objects have their new database IDs
PreserveInsertOrder = true	Items stay in the same order (item[0] is still item[0])

> **Warning:** Always wrap bulk operations in a transaction! If something fails halfway, you want to undo everything, not be left with partial data.

Part II: Keeping Bad Guys Out

Security doesn't have to be scary!

Chapter 6: Roles & Permissions

Who Can Do What

> **Analogy:** In an office, the CEO can go anywhere. Managers can access their floor. Interns can only use the break room. You don't give everyone individual keys - you give them a ROLE (CEO, Manager, Intern) and the role determines access!

The Bad Way: Boolean Flags

```
public class User
{
    public bool CanCreatePosts { get; set; }
    public bool CanDeletePosts { get; set; }
    public bool CanManageUsers { get; set; }
    public bool CanViewReports { get; set; }
    // 20 more flags...
}
```

Problems: 20 flags = 20 database columns. Want to give 50 users "Manager" access? Update 50 records x 20 flags!

The Good Way: Roles

```
User -> has Roles -> ["Manager", "Editor"]
Role -> has Permissions -> ["posts.create", "posts.delete"]

// To check access:
if (user.HasPermission("posts.delete"))
{
    // Show delete button
}
```

> **Key Point:** Change permissions for the Role, and ALL users with that role are updated instantly. No need to update hundreds of user records!

Chapter 7: Password Security

Why "password123" is Terrible

> **Analogy:** You wouldn't write your safe combination on a sticky note on the safe, right? Storing plain passwords is exactly that. Instead, we use a one-way transformation called "hashing" - like shredding a document. You can verify it matches the original, but you can't un-shred it!

The Golden Rules

* **NEVER** store plain passwords - Always hash them
* **Use BCrypt** - It's intentionally SLOW (that's good!)
* **Salt automatically** - BCrypt adds randomness so identical passwords look different

How It Works

```
// When user CREATES password:
var hash = BCrypt.HashPassword("MySecret123");
// Stored: "$2a$12$LQv3c1yqT..." (unreadable gibberish)

// When user LOGS IN:
var isValid = BCrypt.Verify("MySecret123", storedHash);
// Returns true if it matches, false if not
```

Why BCrypt is Slow (On Purpose!)

Algorithm	Hashes per Second	Time to Crack
MD5 (bad)	180 BILLION	~10 hours
BCrypt (good)	3	70 million years

If a hacker steals your database, they'd need 70 million years to crack BCrypt passwords. MD5? They'd be done by lunch.

Chapter 8: Sessions

Remembering Who You Are

> **Analogy:** At a festival, you show your ticket once to get a wristband. After that, you just flash your wristband to access areas. Sessions are digital wristbands - you log in once, get a token, and use it for all future requests!

The Problem

HTTP is "stateless" - each request is independent. The server has the memory of a goldfish! Without sessions, you'd need to send your password with EVERY click.

The Solution: Session Tokens

```
1. Login -> Server creates random token "abc123"
            -> Stores hash in database (never plain!)
            -> Sends token to browser in a cookie

2. Next request -> Browser sends "abc123" automatically
                -> Server: "Ah, that's Luis! Here's your data"

3. Logout -> Server marks token as invalid
          -> Browser deletes cookie
```

Cookie Security Settings

Setting	What It Prevents
HttpOnly	JavaScript can't steal your cookie
Secure	Only sent over HTTPS (encrypted)
SameSite=Strict	Other websites can't use your cookie

Chapter 9: CSRF Protection

Stopping Sneaky Attacks

> **Analogy:** Imagine a website pretends to be your bank, tricks you into clicking a button, and your REAL bank thinks it's you because your cookies are automatically sent! CSRF (Cross-Site Request Forgery) is exactly this attack. Antiforgery tokens are like a secret handshake that proves the request is legit.

How the Attack Works

```
1. You're logged into your-bank.com (cookie saved)
2. You visit evil-site.com
3. Evil site has hidden form:
   <form action="your-bank.com/transfer">
     <input name="amount" value="10000">
     <input name="to" value="hacker">
   </form>
4. Your browser sends your bank cookie automatically!
5. Bank thinks YOU made the transfer
```

The Solution

Antiforgery tokens are like a secret code that only YOUR website knows. Evil site can't read it!

```
// Your real form has a secret token:
<form method="post">
    <input type="hidden" name="__RequestVerificationToken" value="xyz789">
    ...
</form>

// Server checks: "Does the token match?"
// Real site: Yes -> Process request
// Evil site: No token -> BLOCKED!
```

> **Key Point:** Always use [ValidateAntiForgeryToken] on forms that DO something (transfer money, delete data, change settings). Read-only pages don't need it.

Chapter 10: Claims

Your Digital ID Card

> **Analogy:** Your driver's license has "claims" about you: Name, DOB, Address, whether you can drive motorcycles. Claims in .NET are the same - facts about a user. The [Authorize] attribute just checks if you have the right claims!

What Are Claims?

```
// Claims are just key-value pairs:
new Claim("name", "Luis")
new Claim("email", "luis@example.com")
new Claim("role", "Admin")        // Can have multiple!
new Claim("role", "Editor")       // <- Another role
new Claim("permission", "users.delete")  // Custom claims too!
```

The Big Secret

> **Key Point: You don't need ASP.NET Identity!** Many tutorials make you think you need all those AspNetUsers, AspNetRoles tables. Nope! [Authorize] and <AuthorizeView> just check Claims. You can build claims yourself from YOUR user system!

```
// All authorization does is check claims:
[Authorize(Roles = "Admin")]
// Translates to: "Does user have a claim where type=role and value=Admin?"

user.IsInRole("Admin")
// Same thing: checks for role claim

// Build your own claims from your User:
var claims = new List<Claim>
{
    new(ClaimTypes.Name, myUser.UserName),
    new(ClaimTypes.Role, "Admin"),  // One per role!
    new(ClaimTypes.Role, "Editor"),
};
var identity = new ClaimsIdentity(claims, "MyAuth");
var principal = new ClaimsPrincipal(identity);
// Now [Authorize] works!
```

Part III: Blazor Magic

Blazor-specific gotchas and solutions

Chapter 11: Protected Storage

Your Browser's Secret Vault

> **Analogy:** Regular localStorage is like a filing cabinet anyone can open. ProtectedLocalStorage is a safety deposit box - encrypted and tamper-proof!

```
@inject ProtectedLocalStorage Storage

// Save (encrypted automatically)
await Storage.SetAsync("theme", "dark");

// Load (decrypted automatically)
var result = await Storage.GetAsync<string>("theme");
if (result.Success)
{
    var theme = result.Value;  // "dark"
}
```

The Gotcha: Prerendering!

Blazor Server first renders your page on the server (no browser yet!), then sends HTML to the browser. ProtectedLocalStorage needs the browser, so it CRASHES during this "prerender" phase!

```
// WRONG - Crashes during prerender!
protected override async Task OnInitializedAsync()
{
    var result = await Storage.GetAsync<string>("token");
}

// CORRECT - Wait for browser to be ready
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    if (firstRender)  // Only on FIRST render (browser ready!)
    {
        var result = await Storage.GetAsync<string>("token");
        StateHasChanged();  // Re-render with loaded data
    }
}
```

> **Key Point:** OnAfterRenderAsync is the ONLY safe place for browser-dependent stuff like localStorage, sessionStorage, or any JavaScript calls!

Chapter 12: Localization

Speaking Your User's Language

> **Analogy:** Resource files are like phrase books. English book: "Login" = "Login". Portuguese book: "Login" = "Entrar". You write the KEY once, and .NET picks the right translation!

Setup

```
YourProject/
|-- SharedResources.cs           # Empty "marker" class (in root!)
+-- Resources/
    |-- SharedResources.resx         # English (default)
    +-- SharedResources.pt-PT.resx   # Portuguese
```

Using It

```
@inject IStringLocalizer<SharedResources> L

<MudButton>@L["Login"]</MudButton>

// If user's culture is "en" -> "Login"
// If user's culture is "pt-PT" -> "Entrar"
```

> **Warning:** Common Mistake: Putting SharedResources.cs INSIDE the Resources folder. It must be in the PROJECT ROOT! .NET uses the file location to find translations.

Chapter 13: JavaScript Interop

When C# Needs JS Help

> **Analogy:** Blazor runs C#, but sometimes you need JavaScript (DOM manipulation, browser APIs, third-party libraries). JS Interop is the bridge between the two worlds!

```
@inject IJSRuntime JS

// Call JavaScript from C#:
var width = await JS.InvokeAsync<int>("getWindowWidth");

// JavaScript side:
window.getWindowWidth = function() {
    return window.innerWidth;
};
```

The Gotcha (Again): Prerendering!

Just like ProtectedLocalStorage, JS Interop needs the browser. No browser during prerender = crash!

```
// WRONG
protected override async Task OnInitializedAsync()
{
    await JS.InvokeVoidAsync("console.log", "Hi!");  // CRASH
}

// CORRECT
protected override async Task OnAfterRenderAsync(bool firstRender)
{
    if (firstRender)
    {
        await JS.InvokeVoidAsync("console.log", "Hi!");  // Works!
    }
}
```

Blazor Server Timeline

```
Phase 1: PRERENDER (Server)     -> No JS available
Phase 2: HTML sent to browser   -> Still no JS
Phase 3: SignalR connects       -> JS available!
         OnAfterRenderAsync fires here!
```

Cheat Sheets

Security Must-Dos

✓ Do This	✗ Not This
BCrypt for passwords	MD5, SHA, or plain text
Hash session tokens	Store plain tokens
[ValidateAntiForgeryToken]	Skip CSRF protection
"Invalid credentials"	"Email not found" or "Wrong password"
Lockout after 5 fails	Unlimited login attempts

Blazor Lifecycle

Method	Browser Ready?	Use For
OnInitializedAsync	No	Load data from database
OnAfterRenderAsync	Yes	JS interop, localStorage
OnParametersSetAsync	No	React to parameter changes

Quick Decisions

If You Need To...	Use This
Share properties across entities	Base classes
Report success/failure from services	ServiceResult pattern
Import 50+ records fast	BulkExtensions
Make module work with different user types	Adapter pattern
Check if user can do something	Claims + HasPermission()

Access browser storage safely	OnAfterRenderAsync

You made it! These concepts might seem overwhelming now, but they'll become second nature. The best way to learn? Build something! Start small, make mistakes, and iterate.

Remember: Every expert was once a beginner. Keep coding!